Reviewer Pack — Minimal Rank of Noncommutative Algebras and Resolution Proof…

Entry 55a39304130f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 02:05:55 UTC

Minimal Rank of Noncommutative Algebras and Resolution Proof Width

Entry ID: `55a39304130f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 02:05:55 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra (Hopf algebras) **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> The minimal rank of the noncommutative algebra generated by the variables in a CNF φ is linearly correlated with its resolution proof width w(φ), such that min_rank(A(φ)) = Θ(w(φ)) for a Hopf algebra A(φ) associated with φ.

Rationale (proposer's reasoning):

> Noncommutative algebras, particularly Hopf algebras, provide a framework to represent the logical structure of Boolean functions. The minimal rank of a noncommutative algebra could capture the complexity inherent in the resolution proof process, potentially providing insights into the complexity of SAT and its variants.

Taxonomy category: `Hopf_algebras_Boolean_Satisfiability` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `74c12c6f0fed951b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all 30 random seeds, the ratio of the minimal rank to the resolution proof width lies within [0.5, 2] with a mean difference less than or equal to 1.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal rank noncommutative algebras AND resolution proof complexity` - `Hopf algebra associated CNF linearly correlated resolution proof width` - `Resolution proof width 0(min_rank noncommutative algebra CNF)`

Top relevant hits considered: - [http://arxiv.org/abs/math/0610043v5] Lecture Notes on Noncommutative Algebraic Geometry and Noncommutative Tori - [http://arxiv.org/abs/1410.5374v2] Cluster algebras of infinite rank as colimits - [http://arxiv.org/abs/1607.00443v1] Algebraic Proof Complexity: Progress, Frontiers and Challenges - [http://arxiv.org/abs/math/0410471v1] Word Hopf algebras - [http://arxiv.org/abs/0905.2613v3] Categorical Constructions for Hopf Algebras - [http://arxiv.org/abs/1203.2454v2] Crossed product of Hopf algebras - [http://arxiv.org/abs/0705.1265v2] A noncommutative Bohnenblust-Spitzer identity for Rota-Baxter algebras solves Bogoliubov's recursion - [http://arxiv.org/abs/1605.06944v3] Computing minimal free resolutions of right modules over noncommutative algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=241.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                continue
            clauses.append(clause)
        return clauses

    def resolution_width(clauses):
        queue = list(clauses)
        seen = set()
        while queue:
            literal, rest = queue.pop(0)
            seen.add(literal)
            new_clauses = []
            for clause in rest:
                if -literal in clause:
```

```
                    new_clause = [l for l in clause if l != -literal]
                    if not new_clause:
                        return len(queue) + 1
                    new_clauses.append(new_clause)
                else:
                    new_clauses.append(clause)
            queue.extend(new_clauses)
        return float('inf')

    def hopf_algebra_rank(cnf):
        # Simplified rank calculation for demonstration purposes
        return sum(len(set(abs(l) for l in clause)) for clause in cnf)

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    min_rank = hopf_algebra_rank(cnf)
    w_phi = resolution_width(cnf)

    if w_phi == float('inf'):
        return {
            "metric_name": "min_rank_to_w_ratio",
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "resolution_width_infinite"
        }

    ratio = min_rank / w_phi
    return {
        "metric_name": "min_rank_to_w_ratio",
        "metric_value": ratio,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": 0.5 <= ratio <= 2 and abs(ratio - 1) <= 1,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if 0.5 <= r["metric_value"] <= 2 and abs(r["metric_v

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
```

```
        print(f"RESULT: FALSIFIED counterexample=\"min_rank_to_w_ratio_out_of_bounds\" first_faili
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required calculations to verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the allocated time.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14422
2	preregistration	ollama_remote	glm4:latest	0	0	12927
3	novelty	ollama_remote	glm4:latest	0	0	8715
4	novelty	ollama_remote	glm4:latest	0	0	15783
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	105372
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8250
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8391
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32159
9	judge	ollama_remote	glm4:latest	0	0	55486

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 261506 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/55a39304130f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/55a39304130f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/55a39304130f.tar.gz` (if generated)